

2026 控制算法组大作业

——第一期 · MuJoCo 轮腿机器人仿真——

编写：kaixing 时间：2026 年 9 月

1 引言 (Introduction)

很高兴大家对 **RoboWalker 控制算法组** 感兴趣！进入 26 赛季，控制算法组的工作主要围绕两方面展开：**LQR 参数计算与仿真验证**。目前，LQR 计算的代码任务已基本收尾；而仿真验证——尤其是随着强化学习的引入而到来的大量新工作。

因此，本期考核的内容将是一次基于 **MuJoCo** 的仿真实现。希望大家玩得开心。

2 背景 (Background)

本次考核所使用的仿真平台为 **MuJoCo**。MuJoCo 是一个轻量而高效的物理仿真引擎：官方程序包中封装了仿真开发所需的完整接口——前端包括可视化窗口与键鼠交互，后端包括模型关节中的传感器、驱动器（扭矩）等。通过调用这些接口，我们可以借助 **MJCF** 或 **URDF** 模型文件，进行自由的可视化仿真。

2.1 MuJoCo 的安装

MuJoCo 自 2021 年被 DeepMind 收购后已完全开源，官方提供预编译的二进制文件与 Python 绑定，安装十分简便。下面分别介绍在 Ubuntu (Linux) 与 Windows 系统下的安装方法。

2.1.1 Ubuntu 下的安装

在 Ubuntu 下，推荐使用 `pip` 直接安装官方预编译的 Python 包，无需手动编译源码。安装前请确保系统已安装 Python 3.8 及以上版本，并已正确配置 `pip`。

1. 安装 MuJoCo Python 包

在终端中执行：

```
pip install mujoco
```

该命令会自动下载并安装与当前系统架构（x86_64 或 aarch64）匹配的预编译包。

2. 验证安装

安装完成后，在 Python 中运行下面的代码进行验证：

```
import mujoco
import numpy as np
model = mujoco.MjModel.from_xml_string(
    "<mujoco><worldbody><geom type='sphere' size='0.1' /></worldbody></mujoco>")
data = mujoco.MjData(model)
mujoco.mj_step(model, data)
print("MuJoCo 安装成功, 版本:", mujoco.__version__)
```

若能正常输出版本号, 则说明安装成功。

3. (可选) 安装渲染依赖

若需要使用可视化窗口, MuJoCo 依赖 EGL/OpenGL 环境。大多数桌面版 Ubuntu 已自带所需库; 在无头(服务器)环境下, 可补充安装 EGL 支持:

```
sudo apt update
sudo apt install libegl1 libgl1 libosmesa6 libglfw3
```

2.1.2 Windows 下的安装

在 Windows 下, 同样推荐使用 pip 安装官方预编译包。请先确认已安装 Python 3.8 及以上版本, 并将其添加到系统环境变量 PATH 中。

1. 安装 MuJoCo Python 包

打开命令提示符 (CMD) 或 PowerShell, 执行:

```
pip install mujoco
```

2. 验证安装

在 Python 中运行与 Ubuntu 部分相同的验证代码, 确认能够正常导入并输出版本号即可。

3. (可选) 渲染依赖

Windows 下 MuJoCo 使用 OpenGL 进行渲染, 通常无需额外安装。若出现渲染问题, 请确保显卡驱动已更新至最新版本。

4. (可选) 直接使用官方可执行文件

官方还提供了独立的可执行文件 `simulate.exe` (位于 GitHub Releases 页面), 下载后无需 Python 环境即可直接打开 MJCF/URDF 模型进行交互式仿真, 适合快速预览模型。

提示 1 安装出错怎么办? 若安装过程中出现问题, 可参考知乎专栏文章《MuJoCo 教程 01: MuJoCo 入门与环境搭建》(作者: 萧然) : <https://zhuanlan.zhihu.com/p/2065810143070234452>, 也可以直接向 AI 求助。

提示 2 推荐使用 Ubuntu 搭建。 最好在 Ubuntu 环境下完成搭建; 若在 WSL 中遇到渲染问题, 可请 AI 协助下载并安装相关插件解决。

3 考核任务 (Task)

本期作业的内容，是基于 LQR 算法实现一个简化轮腿机器人的仿真。该机器人拥有两个髋关节和两个轮关节，具体的模型可查看文件夹中的建模部分。

3.1 LQR

整个作业大致可分为两个阶段。

第一阶段：理解 LQR 算法。大家可以从网络上搜索相关知识进行学习，这里推荐 DR_CAN 老师的视频讲解，以及他的著作《控制之美》。

对本任务的模型而言，其状态空间方程具有一定的复杂度，而这部分计算又不是本次仿真的重点，因此我们提供了具体的 LQR 计算代码框架。不过，一方面代码中的数据需要大家自己去建模文件中测量，另一方面代码中参数的调整仍然需要大家对 LQR 算法有足够的了解。

3.2 URDF

第二阶段是仿真实现。MuJoCo 建模一般需要经过 SLDPRT/SLDASM → URDF → MJCF 的流程：原则上 URDF 也可以直接接入 MuJoCo，但由于其无法支持闭链结构，这里最终选择 MJCF。第一步的转换需要在 SolidWorks 中进行，具体过程可参考知乎文章《笔记：从 SW 中导出 URDF 的逐步骤记录》（作者：陈皮不是大橘）：<https://zhuanlan.zhihu.com/p/2036077117008558078>

需要说明的是，教程中处理的模型往往都经过简化，或本身外观就比较简单；而机械设计给出的原始图纸通常是较为复杂的装配体——若直接导出，会造成模型面数过多，进而导致仿真“爆炸”。因此，我们需要先对每个部分进行简化。本次大作业中的轮子就是一个非常适合拿来练习的对象，具体处理步骤如下：

1. 首先打开轮子的装配体文件 wheel.SLDASM，在顶部找到 工具 → Defeature。

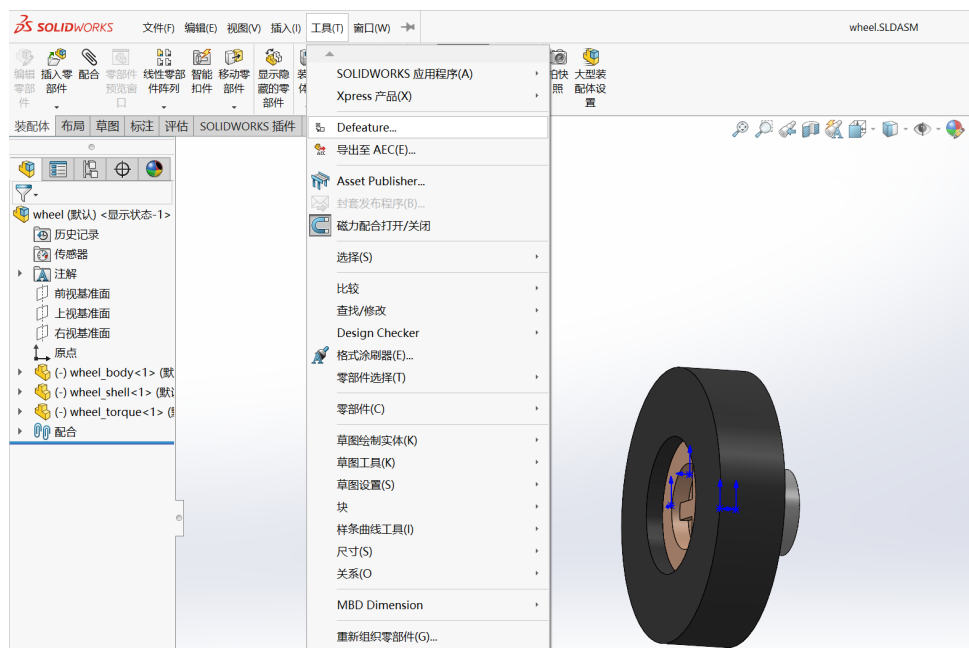


图 1: 步骤 1: 找到 Defeature 入口

2. 点击后，左侧会弹出简化设置面板：在下方“消除特征的方法”处选择简化几何图形，再点击上方蓝色的箭头进入下一步。

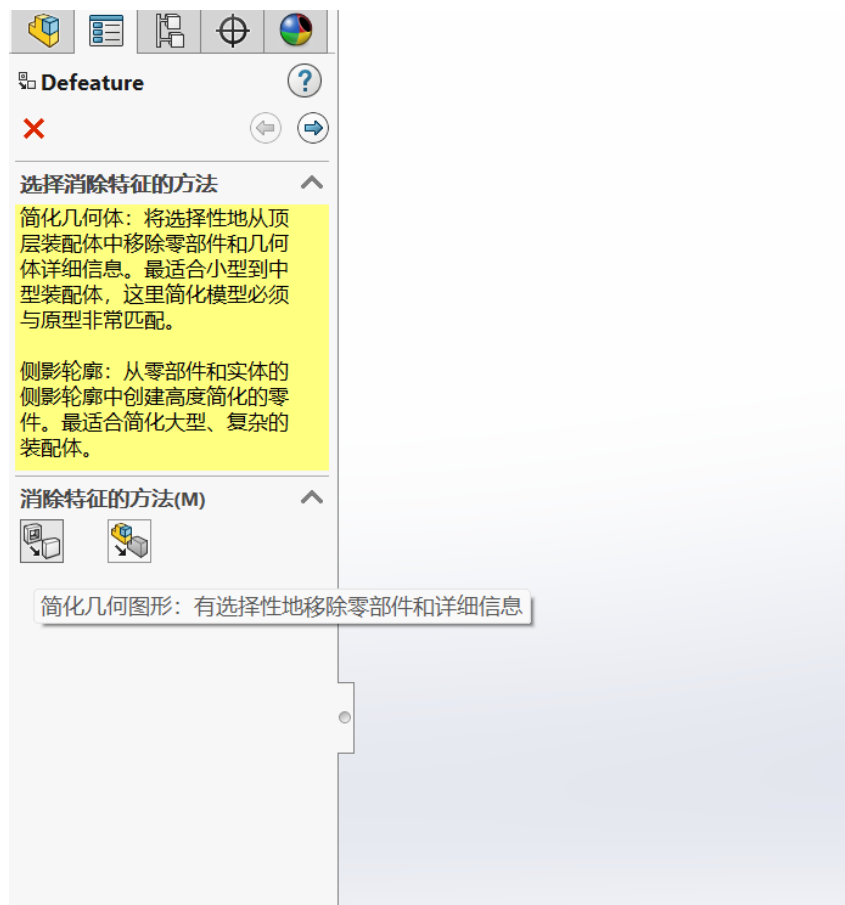


图 2: 步骤 2: 选择简化方法

- 接着按图中所示，勾选**内部零件**与**小型零部件**；小型零部件的去除比例一般设为 0.05，其余参数可视情况调整，然后点击下一步。

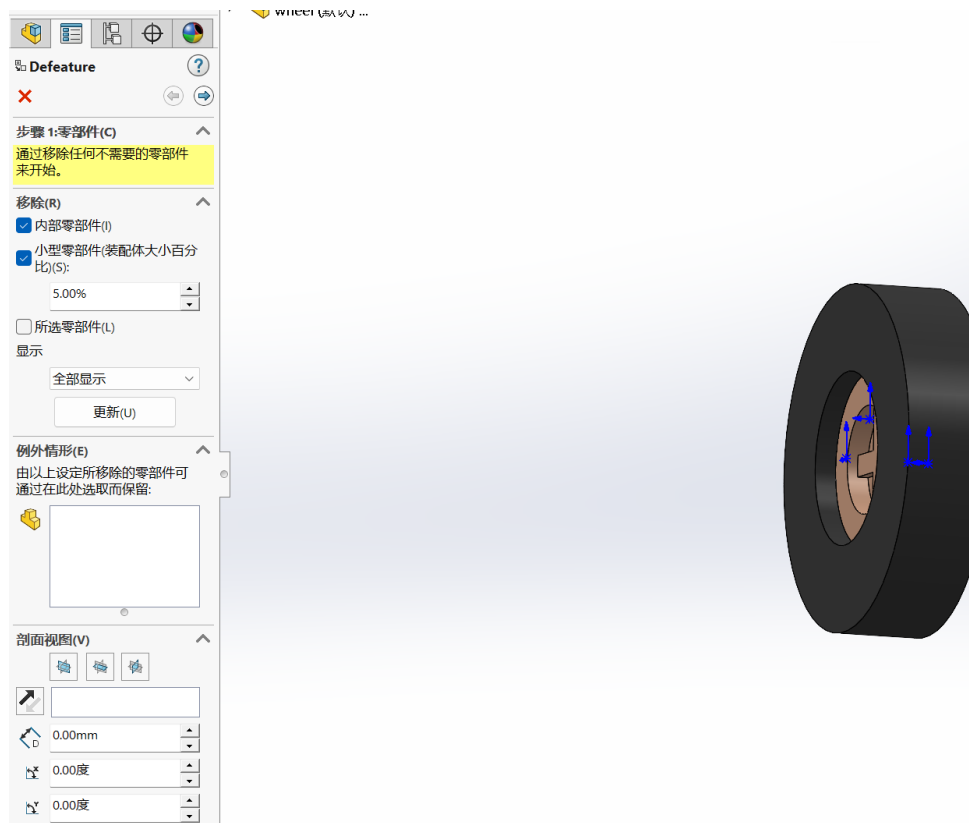
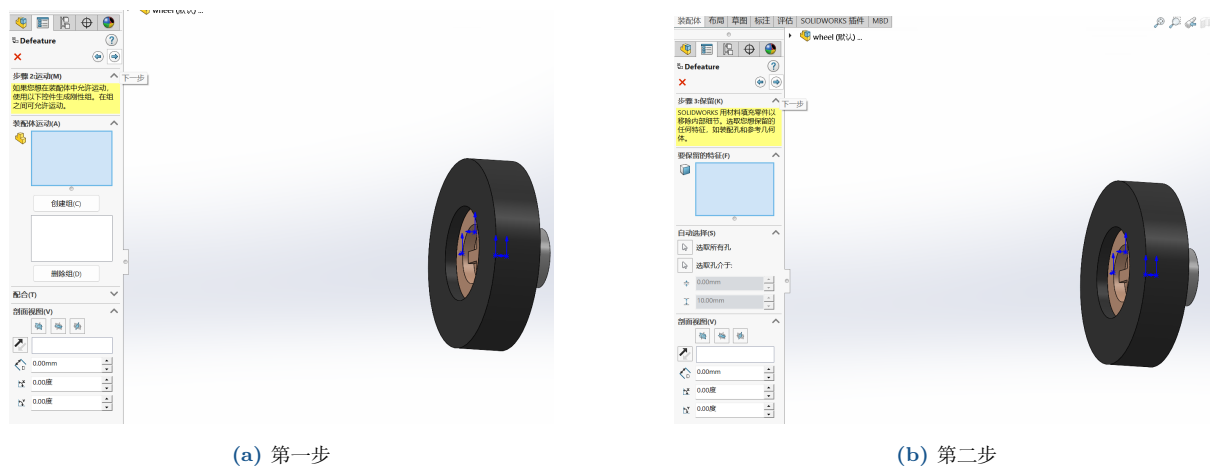


图 3: 步骤 3: 勾选需要去除的零部件

4. 由于我们要简化的都是仿真中的最小组成单元, 并不存在允许运动的情况, 因此接下来的两步可以直接点击“跳过”:



(a) 第一步

(b) 第二步

图 4: 步骤 4: 直接跳过运动相关步骤

5. 等待简化完成后, 会生成类似下面的预览界面, 简单检查若无问题即可点击下一步。

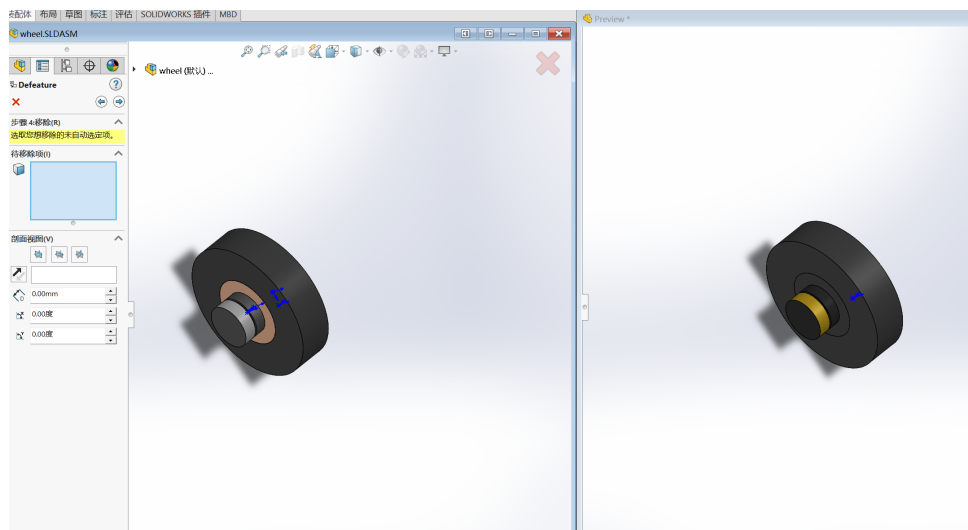


图 5: 步骤 5: 简化结果预览

6. 最后一步务必选择**保存为新文档**，随后点击对号确认，即可生成一个简化后的新零件。

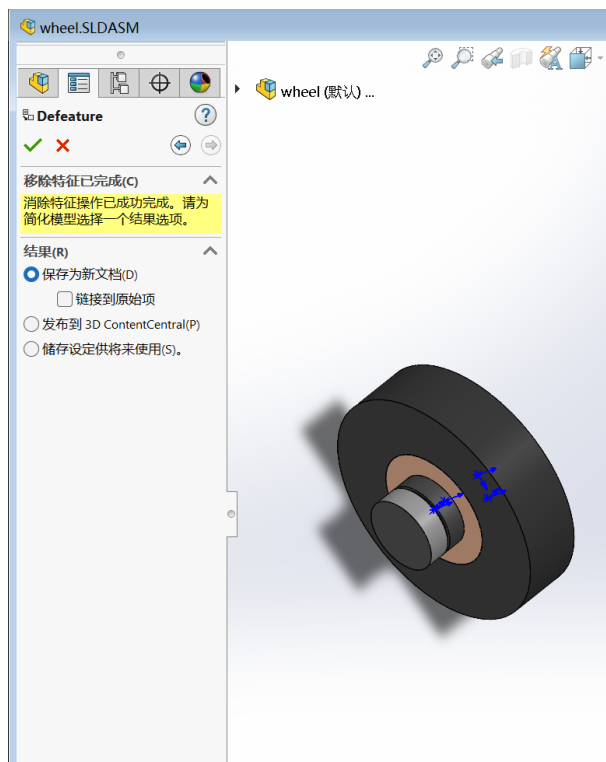


图 6: 步骤 6: 保存为新文档

3.3 MJCF

MJCF 本身是通过 XML 文件对从 SolidWorks 中导出的各个视觉模型文件进行调用,并在 XML 中为每个模块规定相应的动力学属性。从 URDF 到 MJCF 的工作,本质上就是编写 XML 文件——因此,无论采用官方方法还是交给 AI 处理,这部分都方便快捷。对 MJCF 文件结构与功能的了解,实际上就是对 MuJoCo 引擎本身的了解,相关内容就留给大家自行探索。

3.4 仿真控制

MuJoCo 同时提供 Python 与 C++ 接口；但为了与实车代码保持一致，我们的仿真基本都选用 C++ 进行搭建。整个仿真工程的文件结构，比较推荐如下形式：

- **接口文件**——负责将 *MuJoCo* 中的传感器和电机接口，与控制代码中的角度读取、扭矩输出对接；
- **运行文件**——负责运行 *MuJoCo* 仿真的可视化渲染、键鼠交互与控制循环；
- **控制文件**——负责控制代码的运行：通过接口文件读取实时发送来的数据，并向接口文件输出扭矩。

单独把控制文件拿出来，在本任务中看起来似乎有些多余——控制逻辑简单，还要因此多出一个接口文件，显得十分麻烦。但这样设计是有原因的：本任务的控制过程虽然比较简单，但在实车仿真中，控制文件需要承接整个车辆下位机中的控制代码，因此把它单独分离出来是十分必要的。

4 后言

具体的结束时间由算法组组长规定，以上题目叙述中若有地方出现错误或歧义，欢迎大家及时找我沟通。